第五章 数组和广义表

- 5.1 数组的定义
- 5.2 数组的顺序表示和实现
- 5.3 矩阵的压缩存储
  - 5.3.1 特殊矩阵
  - 5.3.2 稀疏矩阵
- *5.4 广义表的定义
- *5.5 广义表的存储结构

5.1 数组的定义

在早期的高级语言中，数组是唯一可供使用的数据类型。由于数组中各元素具有统一的类型，并且数组元素的下标一般具有固定的上界和下界，因此，数组的处理比其它复杂的结构更为简单。多维数组是向量的推广。例如，二维数组：

$A_0=(a_{0,0}\ a_{0,1}\ \cdots\ a_{0,n-1})$

$A_1=(a_{1,0}\ a_{1,1}\ \cdots\ a_{1,n-1})$

$\cdots\ \cdots\ \cdots\ \cdots$

$A_{m-1}=(a_{m-1,0}\ a_{m-1,1}\ \cdots\ a_{m-1,n-1})$



$A=(A_0,A_1,\cdots A_{m-1})$

数组的类型定义

ADT Array {

数据对象：

$$D = \{a_{j_1,j_2,\ldots,j_i,j_n} \mid j_i = 0,\ldots,b_i - 1,\ i = 1,2,..,n\}$$

数据关系：

$$R = \{R1, R2, \ldots, Rn\}$$

$$Ri = \{<a_{j_1,\ldots j_i,\ldots j_n}, a_{j_1,\ldots j_i+1,\ldots j_n}> \mid 0 \le j_k \le b_k - 1,$$

$$1 \le k \le n \text{ 且 } k \ne i,\ 0 \le j_i \le b_i - 2,\ i = 2,\ldots,n\}$$

基本操作：

} ADT Array

二维数组的定义：

数据对象：

$$D = \{a_{ij} \mid 0 \le i \le b_1-1,\ 0 \le j \le b_2-1\}$$

数据关系：

$$R = \{ ROW, COL \}$$

$$ROW = \{<a_{i,j}, a_{i+1,j}> \mid 0 \le i \le b_1-2,\ 0 \le j \le b_2-1\}$$

$$COL = \{<a_{i,j}, a_{i,j+1}> \mid 0 \le i \le b_1-1,\ 0 \le j \le b_2-2\}$$

基本操作：

InitArray(&A, n, bound1, ..., boundn)

DestroyArray(&A)

Value(A, &e, index1, ..., indexn)

Assign(&A, e, index1, ..., indexn)

InitArray(&A, n, bound1, ..., boundn)

操作结果： 若维数 n 和各维长度合法，则构造相应的数组A，并返回OK。

DestroyArray(&A)

操作结果：销毁数组A。

Value(A, &e, index1, ..., indexn)

初始条件： A是n维数组，e为元素变量，随后是n 个下标值。

操作结果： 若各下标不超界，则e赋值为所指定的A 的元素值，并返回OK。

Assign(&A, e, index1, ..., indexn)

初始条件： A是n维数组，e为元素变量，随后是n 个下标值。

操作结果： 若下标不超界，则将e的值赋给所指定的A的元素，并返回OK。

5.2 数组的顺序表示和实现

类型特点:

1) 只有引用型操作，没有加工型操作;

2) 数组是多维的结构，而存储空间是一个一维的结构。

有两种顺序映象的方式:

1)以行序为主序(低下标优先);

2)以列序为主序(高下标优先)。

行（列）优先顺序

行优先顺序——将数组元素按行排列，第i+1个行向量紧接在第i个行向量后面。以二维数组为例，按行优先顺序存储的线性序列为：

$a_{0,0}, a_{0,1}, \ldots, a_{0,n-1}, a_{1,0}, a_{1,1}, \ldots a_{1,n-1}, \ldots\ldots, a_{m-1,0}, a_{m-1,1}, \ldots, a_{m-1,n-1}$

PASCAL、C/C++语言，数组是按行优先顺序存储的。

a[i][j] ---- 第i*n+j个位置

列优先顺序——将数组元素按列向量排列，第j+1个列向量紧接在第j个列向量之后，A的m*n个元素按列优先顺序存储的线性序列为：

$a_{0,0}, a_{1,0}, \ldots, a_{m-1,0}, a_{0,1}, a_{1,1}, \ldots a_{m-1,1}, \ldots\ldots, a_{0,n-1}, a_{1,n-1}, \ldots, a_{m-1,n-1}$

FORTRAN语言中，数组是按列优先顺序存储的。

a[i][j] ---- 第j*m+i个位置

多维数组的优先顺序

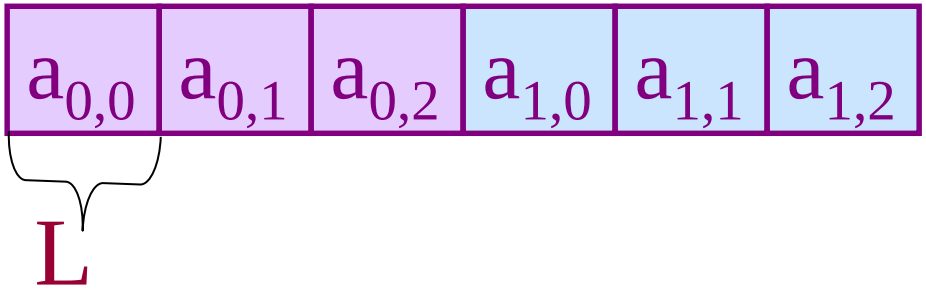
以上规则可以推广到多维数组的情况：行优先顺序可规定为先排最右的下标，从右到左，最后排最左下标：列优先顺序与此相反，先排最左下标，从左向右，最后排最右下标。

按上述两种方式顺序存储的序组，只要知道开始结点的存放地址（即基地址），维数和每维的上、下界，以及每个数组元素所占用的单元数，就可以将数组元素的存放地址表示为其下标的线性函数。因此，数组中的任一元素可以在相同的时间内存取，即顺序存储的数组是一个随机存取结构。

以“行序为主序”的存储映象

例如：

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$



二维数组A中任一元素$a_{i,j}$ 的存储位置

$$LOC(i,j) = LOC(0,0) + (b_2 \times i + j) \times L$$

称为**基地址**或基址。

二维数组A(c_1:d_1,c_2:d_2)(c_i<=d_i, 两个都是整数),按“行优先顺序”存储在内存中，假设每个元素占用d个存储单元。

$$A=\begin{bmatrix} a_{c1,c2}a_{c1,c2+1}\cdots\cdots a_{c1,d2} \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots \\ a_{d1,c2}a_{d1,c2+1}\cdots\cdots a_{d1,d2} \end{bmatrix}$$

元素a_{ij}的存储地址应是数组的基地址加上排在a_{ij}前面的元素所占用的单元数。因此，a_{ij}的地址计算函数为：

$$LOC(a_{ij})=LOC(a_{c1,c2})+(i-c_1)*(d_2-c_2+1)*d+(j-c_2)*d$$

$$= LOC(a_{c1,c2})+i*(d_2-c_2+1)*d-c_1(d_2-c_2+1)*d+j*d-c_2*d$$

命：$M_1=(d_2-c_2+1)*d$, $M_2=d$, 则有

$$LOC(a_{ij})=v_0+i*M_1+j*M_2$$

其中：$v_0=LOC(a_{c1,c2})-c_1*M_1-c_2*M_2$

推广到一般：$A(c_1:d_1,c_2:d_2,\ldots c_n:d_n)$,每个元素占d个字节，以行为主存储，则有

$LOC(a_{i1,i2,\ldots,in})=V_0+i_1*M_1+i_2*M_2+\ldots+i_n*M_n$

其中：

$$\begin{cases} M_j = d \times \prod_{k=j+1}^{n}(d_k - c_k + 1) \\ M_n = d \end{cases}$$

最后一维前有多少个元素

每个元素占用的存储单元

$$V_0 = LOC(a_{c1,c2,\ldots cn}) - \sum_{j=1}^{n} M_j \times C_j$$

命：$M_1=(d_2-c_2+1)*d$，　$M_2=d$，则有

$LOC（a_{ij})=v_0+i*M_1+j*M_2$

其中：$v_0=LOC（a_{c1,c2})-c_1*M_1-c_2*M_2$

5.3 矩阵的压缩存储

在科学与工程计算问题中，矩阵是一种常用的数学对象，在高级语言编制程序时，简单而又自然的方法，就是将一个矩阵描述为一个二维数组。矩阵在这种存储表示之下，可以对其元素进行随机存取，各种矩阵运算也非常简单。但是在矩阵中非零元素呈某种规律分布或者矩阵中出现大量的零元素的情况下，占用了许多空间去存储重复的非零元素或零元素，对高阶矩阵会造成极大的浪费，为了节省存储空间， 可以对这类矩阵进行压缩存储：即只为非零元素分配存储空间；对零元素不分配空间等技术来处理。

何谓稀疏矩阵?

假设 m 行 n 列的矩阵含 t 个非零元素，则称

$$\delta = \frac{t}{m \times n}$$

为**稀疏因子**。

通常认为 $\delta \leq 0.05$ 的矩阵为稀疏矩阵。

以常规方法，即以二维数组表示高阶的稀疏矩阵时产生的**问题**：

1) 零值元素占了很大空间；

2) 计算中进行了很多和零值的运算，遇除法，还需判别除数是否为零。

解决问题的原则：

1) 尽可能少存或不存零值元素；

2) 尽可能减少没有实际意义的运算；

3) 操作方便。 即：

能尽可能快地找到与下标值(i，j)对应的元素，

能尽可能快地找到同一行或同一列的非零值元。

有两类稀疏矩阵：

1) 特殊矩阵

非零元在矩阵中的分布有一定规则

例如: 三角矩阵

对角矩阵

2) 随机稀疏矩阵

非零元在矩阵中随机出现

5.3.1 特殊矩阵

所谓特殊矩阵是指非零元素或零元素的分布有一定规律的矩阵。几种特殊矩阵的压缩存储。

1、对称矩阵 、三角矩阵

在一个n阶方阵A中，若元素满足下述性质：

$a_{ij}=a_{ji}$　　$1 \leqslant i,j \leqslant n$

则称A为对称矩阵。

对称矩阵中的元素关于主对角线对称，故只要存储矩阵中上三角或下三角中的元素，这样，能节约近一半的存储空间。不失一般性，我们按“行优先顺序”存储主对角线（包括对角线）以下的元素。

$$
\begin{pmatrix}
a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & \cdots & \cdots & \cdots & a_{1,n-1} & a_{1,n} \\
a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & \cdots & \cdots & \cdots & \cdots & \\
a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & \cdots & \cdots & \cdots & \cdots & \\
a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & \cdots & \cdots & \cdots & \cdots & \\
a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & \cdots & \cdots & \cdots & \\
\cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \\
a_{n-1,1} & \cdots & \cdots & \cdots & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} & & & \\
a_{n,1} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & a_{n,n-1} & a_{n,n} &
\end{pmatrix}
$$

一个n阶的对称阵（三角阵）可用一个容量是（n+1)n/2的一维数组存储

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	..	a_{nn}

n阶对称阵与一维数组空间对应关系：

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & \cdots & \cdots & \cdots & a_{1,n-1} & a_{1,n} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} & \cdots & \cdots & \cdots & \cdots & \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & \cdots & \cdots & \cdots & \cdots & \\ a_{41} & a_{42} & a_{43} & a_{44} & a_{45} & \cdots & \cdots & \cdots & \cdots & \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} & a_{56} & \cdots & \cdots & \cdots & \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-1,1} & \cdots & \cdots & \cdots & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} & & & \\ a_{n,1} & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & a_{n,n-1} & a_{n,n} & \end{pmatrix}$$

$$\underbrace{(\ n^2}_{\text{总数}} \ - \ \underbrace{n}_{\text{对角线元素}} \ ) \ / \ \underbrace{2}_{\text{取一半元素}} \ + \ \underbrace{n}_{\text{对角线元素}} \ = \ (n+1)n/2$$

一个n阶的对称阵（三角阵）可用一个容量是（n+1)n/2的一维数组存储

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	..	a_{nn}

n阶对称阵与一维数组空间对应关系：

a_{11} a_{12} a_{13} a_{14} a_{15} …. …. ….$a_{1,n-1}$ $a_{1,n}$
a_{21} a_{22} a_{23} a_{24} a_{25} …. …. …. ….
a_{31} a_{32} a_{33} a_{34} a_{35} …. …. …. ….
a_{41} a_{42} a_{43} a_{44} a_{45} …. …. …. ….
a_{51} a_{52} a_{53} a_{54} a_{55} a_{56} …. ….. ….
…. …. …. …. …. …. …. …. ….
$a_{n-1,1}$ …. …. …. $a_{n-1,n-2}$ $a_{n-1,n-1}$ $a_{n-1,n}$
$a_{n,1}$ …. …. …. …. …. …. $a_{n,n-1}$ $a_{n,n}$

下三角

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	a_{41}	a_{42}	a_{43}	a_{44}	..	..	$a_{n,1}$	..	..	$a_{n,n}$
1	2	3	4	5	6	7	8	9	10	..	..	(n-1)n/2	..	..	(n+1)n/2

1 + 2 + 3 + 4 + n =（n+1)n/2

一个n阶的对称阵（三角阵）可用一个容量是（n+1)n/2的一维数组存储

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}	a_{33}	..	a_{nn}

A中任意一元素 a_{ij}与它的存储位置k之间存在着如下对应关系：

前i-1行的元素个数

$$k = \begin{cases} i(i-1)/2+j-1 & \text{当 } i \geq j \\ j(j-1)/2+i-1 & \text{当 } i < j \end{cases}$$ （对称）

K=0, 1, 2, …, ((n+1)n/2)-1

1+2+3+(i-1) = (i-1)i/2

(i-1)i/2 + j = k + 1　　i ≥ j 且i, j ≥ 1

3、对角矩阵

对角矩阵中，所有的非零元素集中在以主对角线为中心的带状区域中，即除了主对角线和主对角线相邻两侧的若干条对角线上的元素之外，其余元素皆为零。下图给出了一个三对角矩阵。

$$
\begin{pmatrix}
a_{11} & a_{12} & & & & \\
a_{21} & a_{22} & a_{23} & & & \\
 & a_{32} & a_{33} & a_{34} & & \\
 & & \cdots & \cdots & \cdots & \\
 & & & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\
 & & & & a_{n,n-1} & a_{n,n}
\end{pmatrix}
$$

也可以采用某个原则（或以行为主，或以对角线的顺序）将其压缩存储到一维数组上。

但还有另外一种矩阵，非零元较零元少，且分布没有一定规律，称为稀疏矩阵。

随机稀疏矩阵的压缩存储方法:

一、三元组顺序表

二、行逻辑联接的顺序表(**)

三、十字链表(**)

一、三元组顺序表

在存储稀疏矩阵时，为了节省存储单元，不存储0元素。

由于非零元素的分布一般是没有规律的，因此在存储非零元素的同时，必须同时记下它所在的行和列的位置（i,j)。

所以，一个三元组(i,j,a_{ij})唯一确定了矩阵A的一个非零元。因此，稀疏矩阵可由表示非零元的三元组及其行列数唯一确定。

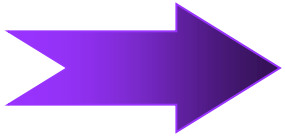
$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{pmatrix}$$

TSMatrix M;

		i	j	e
M.data	0	1	2	12
	1	1	3	9
	2	3	1	-3
	3	3	6	14
	4	4	3	24
	5	5	2	18
	6	6	1	15
	7	6	4	-7
M.mu			6	
M.nu			7	
M.tu			8	

如何求转置矩阵?

$$\begin{bmatrix} 0 & 14 & 0 & 0 & -5 \\ 0 & -7 & 0 & 0 & 0 \\ 36 & 0 & 0 & 28 & 0 \end{bmatrix}$$



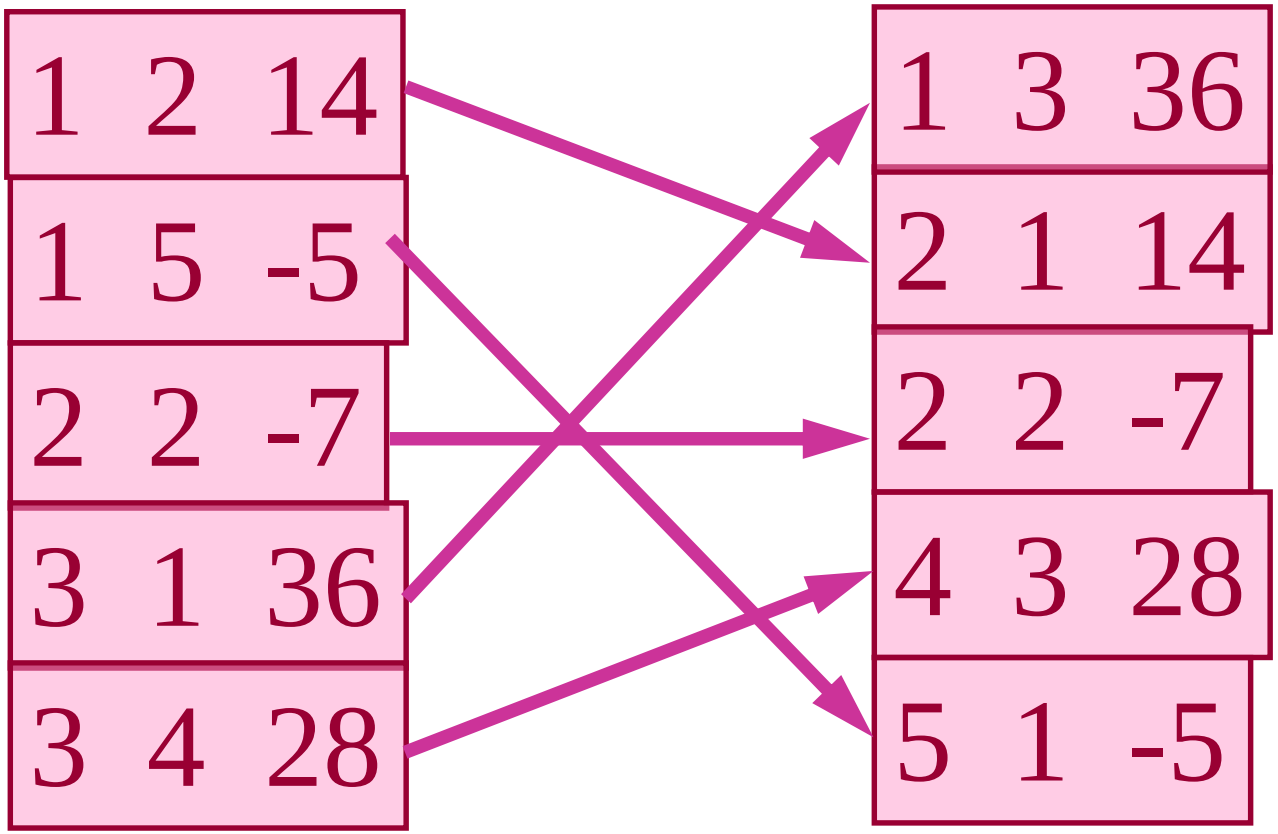
$$\begin{bmatrix} 0 & 0 & 36 \\ 14 & -7 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 28 \\ -5 & 0 & 0 \end{bmatrix}$$

用常规的二维数组表示时的算法

```
for (col=1; col<=nu; ++col)
    for (row=1; row<=mu; ++row)
        T[col][row] = M[row][col];
```

其时间复杂度为: O(mu×nu)

用“三元组”表示时如何实现?



转置运算算法基本思想

- 对 M.data从头至尾扫描：
  - 第一次扫描时，将M.data中列号为1的三元组赋值到T.data中
  - 第二次扫描时，将M.data中列号为2的三元组赋值到T.data中
  - ……
  - 依此类推，直至将M.data所有三元组赋值到T.data中

```
TransposeSMatrix(M, T){ //求M的转置，用T保存
    T.mu=M.nu;  T.nu=M.mu;  T.tu=M.tu;
    if (T.tu)  {
      q=0;  // q为当前三元组在T.data[ ]存储位置(下标)
      for (col=1; col<=M.nu;  ++col)//扫描的顺序
       for (p=0;  p<=M.tu-1;  ++p)//每次扫描的长度
         //p为扫描M.data[ ]的“指示器”
         if (M.data[p].j= =col){
            T.data[q].i =M.data[p].j;
            T.data[q].j=M.data[p].i;
            T.data[q].e=M.data[p].e; ++q;}
     }
}//  TransposeSMtrix
```

转置运算算法分析

- 分析这个算法，主要的工作是在p和col的两个循环中完成的，故算法的时间复杂度为O(nu*tu)，即矩阵的列数和非零元的个数的乘积成正比。而一般传统矩阵的转置算法的时间复杂度为O(nu*mu)。当非零元素的个数t和mu*nu同数量级时，算法transmatrix的时间复杂度为$O(mu*nu^2)$。
- 三元组顺序表虽然节省了存储空间，但时间复杂度比一般矩阵转置的算法还要复杂，同时还有可能增加算法的难度。因此，此算法仅适用于t<<m*n的情况。

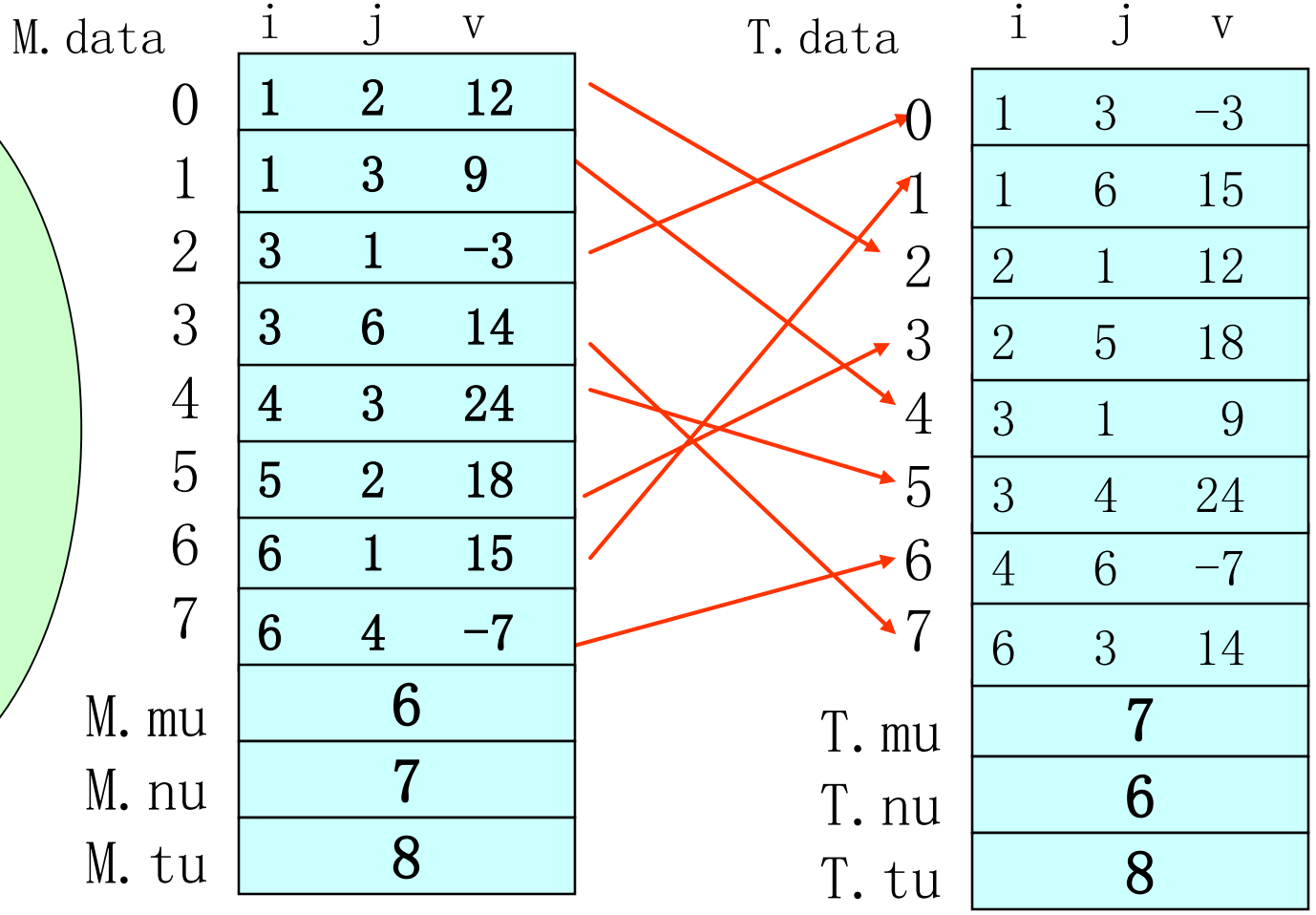
快速转置算法的基本思想

- 下面给出另外一种称之为快速转置的算法，其算法思想为：对A扫描一次，按A第二列提供的列号一次确定位置装入B的一个三元组。

- 具体实施如下：一遍扫描先确定三元组的位置关系，二次扫描由位置关系装入三元组。可见，位置关系是此种算法的关键。

例子

各列第一个非零元三元组按先前求出的位置，放至T.data中



各列后续的非零元三元组，顺序放到对应列元素的后面

辅助向量

为先求得M各列第一个非零元三元组在T.data中的位置。引入**两个辅助向量num[ ]、pos[ ]**：

num[col]：存储第col列非零元个数

pos[col]：存储第col列第一个非零元三元组在T.data中的位置，pos[col]的计算：

$$\begin{cases} pos[1]=1 \\ pos[col]=pos[col-1]+num[col-1] \end{cases}$$

2<=col<=n

如果第1列不存在非零元时？

例 矩阵M

i	j	v
1	2	12
1	3	9
3	1	-3
3	6	14
4	3	24
5	2	18
6	1	15
6	4	-7

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	1	3	5	7	8	8	9

快速转置算法主要步骤

- 求M中各列非零元个数num[ ]
- 求M中各列第一个非零元在T.data中的下标pos[ ];
- 对M.data进行一次扫描， 遇到col列的第一个非零元三元组时，按pos[col]的位置，将其放至T.data中，当再次遇到col列的非零元三元组时，只须顺序放到col列元素的后面;

快速转置算法图示1

求各列非零元个数

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]							

求各列第1个非零元在b中位置

M.data

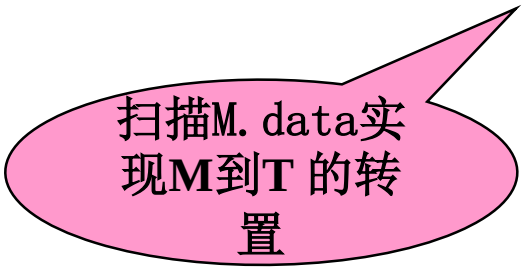
i	j	v
1	2	12
1	3	9
3	1	-3
3	6	14
4	3	24
5	2	18
6	1	15
6	4	-7

T.data

i	j	v

快速转置算法图示2

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	1	3	5	7	8	8	9



i	j	v
1	2	12
1	3	9
3	1	-3
3	6	14
4	3	24
5	2	18
6	1	15
6	4	-7

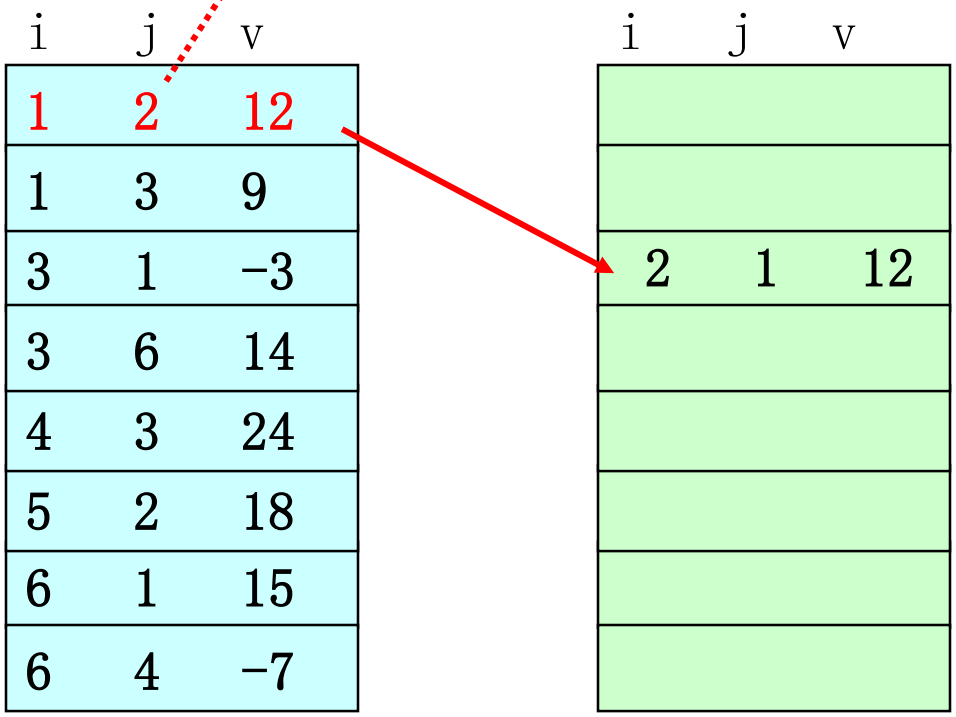
M.data

i	j	v

T.data

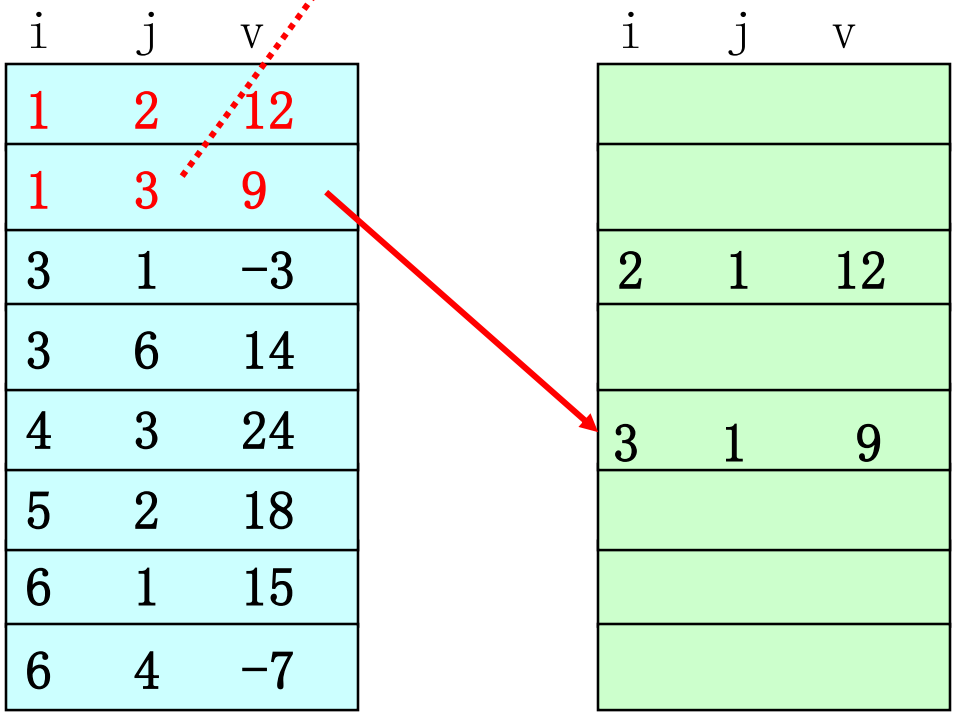
快速转置算法图示3

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	1	3	5	7	8	8	9



快速转置算法图示4

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	1	4	6	7	8	8	9

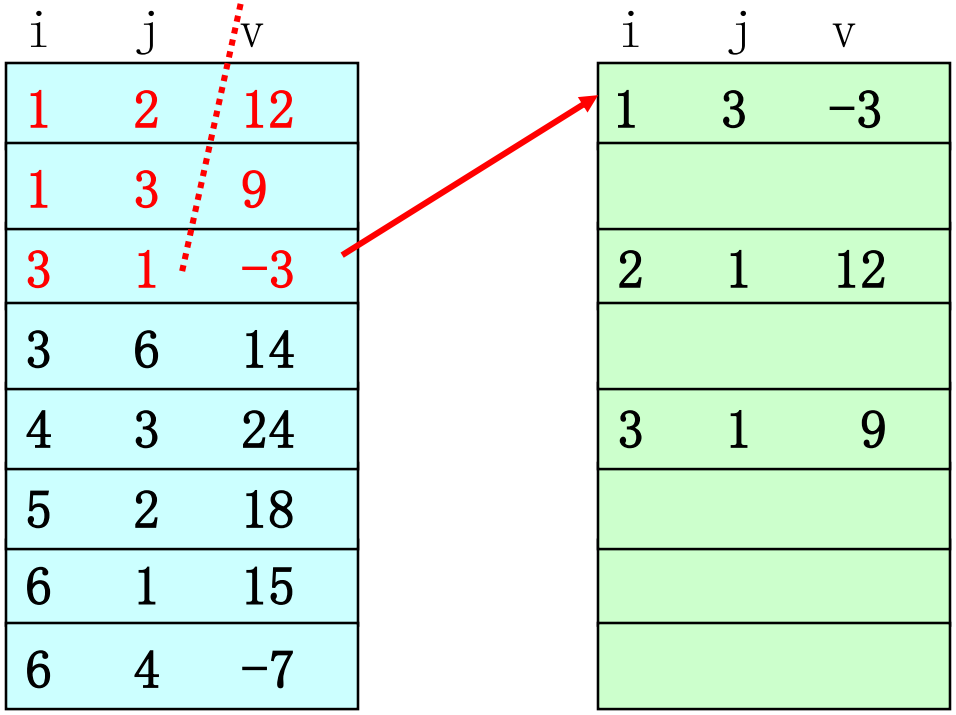


M.data

T.data

快速转置算法图示5

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	1	3	5	6	7	7	8

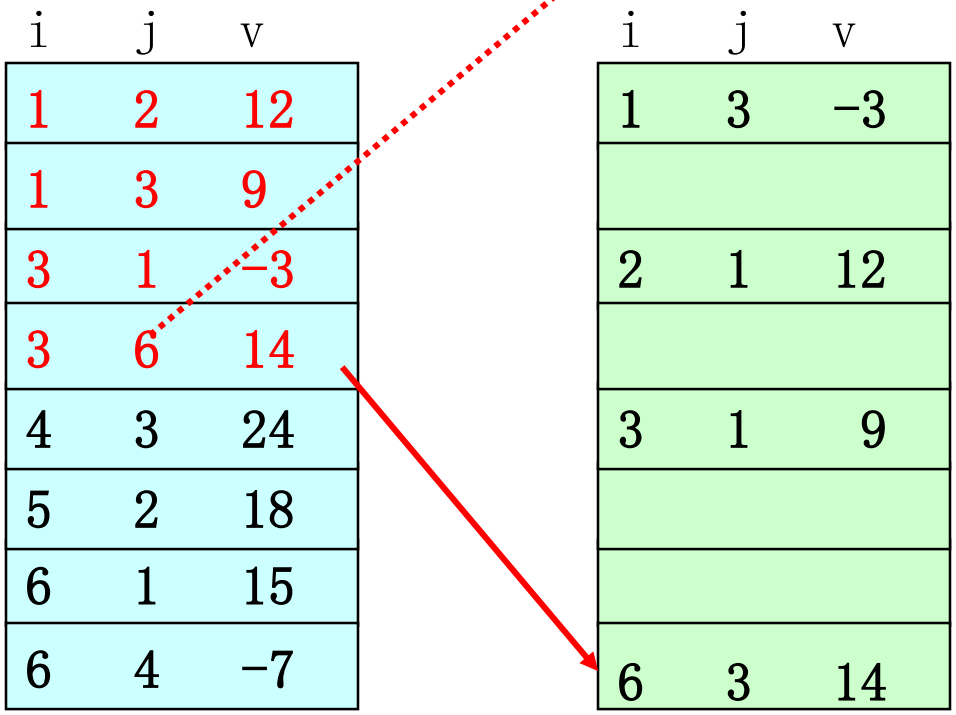


M.data

T.data

快速转置算法图示6

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	2	4	6	7	8	9	9

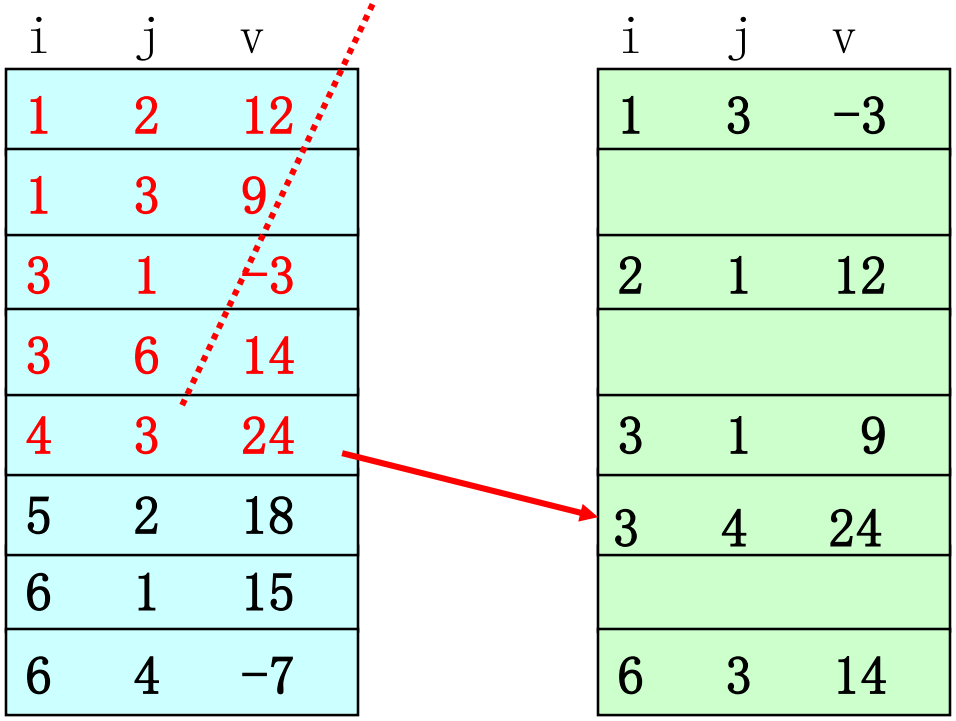


M.data

T.data

快速转置算法图示7

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	2	4	6	7	8	9	9

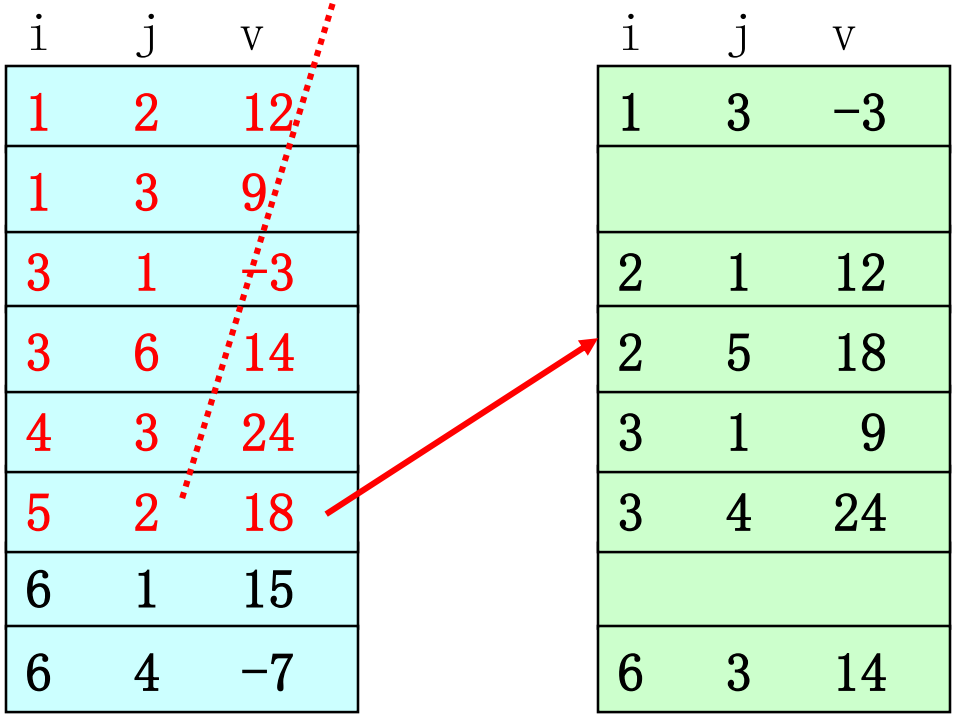


M.data

T.data

快速转置算法图示8

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	2	4	7	7	8	9	9

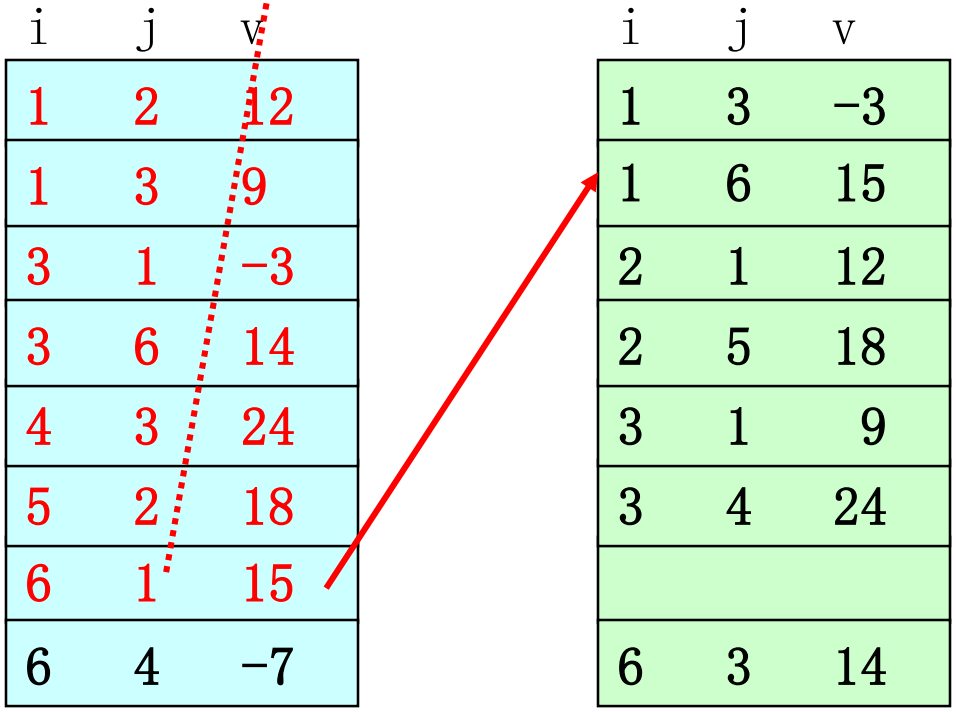


M.data

T.data

快速转置算法图示9

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	3	5	7	7	8	9	9

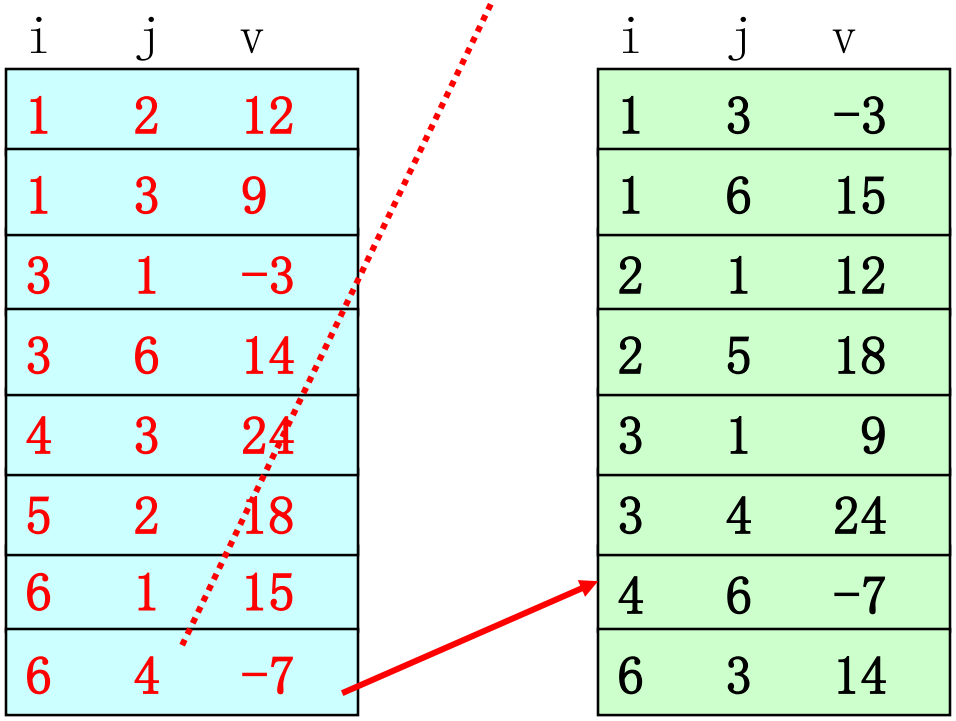


M.data

T.data

快速转置算法图示10

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
pos[col]	3	5	7	8	8	9	9



M.data

T.data

```
Status FastTransposeSMatrix(TSMatrix M, TSMatrix &T){
  T.mu = M.nu;  T.nu = M.mu;  T.tu = M.tu;
  if (T.tu) {
    for (col=1; col<=M.nu; ++col)  num[col] = 0;
    for (t=1; t<=M.tu; ++t)  ++num[M.data[t].j];
    cpot[1] = 1;
    for (col=2; col<=M.nu; ++col)
      cpot[col] = cpot[col-1] + num[col-1];
    for (p=1; p<=M.tu; ++p) {  转置矩阵元素  }
  } // if
  return OK;
} // FastTransposeSMatrix
```

```
Col = M.data[p].j;
q = cpot[col];
T.data[q].i = M.data[p].j;
T.data[q].j = M.data[p].i;
T.data[q].e = M.data[p].e;
++cpot[col];
```

分析算法FastTransposeSMatrix的时间复杂度：

```
for (col=1; col<=M.nu; ++col) … …
for (t=1; t<=M.tu; ++t) … …
for (col=2; col<=M.nu; ++col) … …
for (p=1; p<=M.tu; ++p) … …
```

时间复杂度为: O(M.nu+M.tu)

3)三元组表示矩阵的相乘

由于两个稀疏矩阵相乘结果不一定是稀疏矩阵，所以积用二维数组表示。

设M为m_1行n_1列，N为n_1行n_2列，则它们的积Q为m_1行n_2列，且有：

$$q[i,j]=\sum_{k=1}^{n1} m[i,k]\times n[k,j]$$

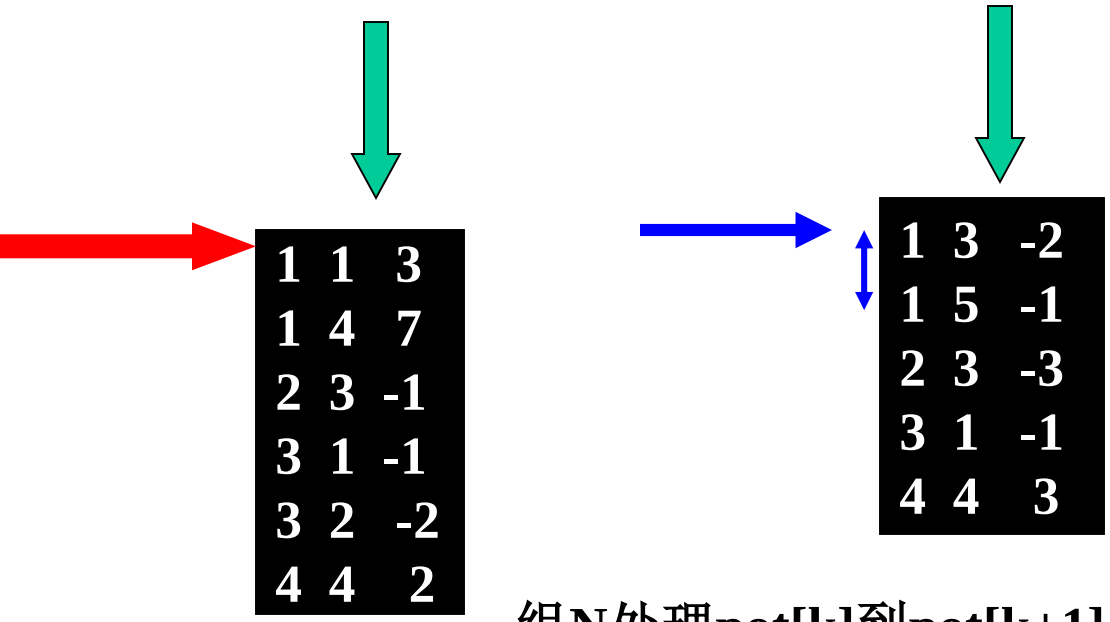
$$\mathbf{M}=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad \mathbf{N}=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix} \Rightarrow \mathbf{Q}=\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & -0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 1 \\ 0 & 0 & 0 & 6 & 0 \end{pmatrix}$$

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



2）形成矩阵N各行的num, pot

row	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为N的行号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&0&0&0\\0&0&0&0&0\\0&0&0&0&0\\0&0&0&0&0\end{pmatrix}$$

3*(-2)

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

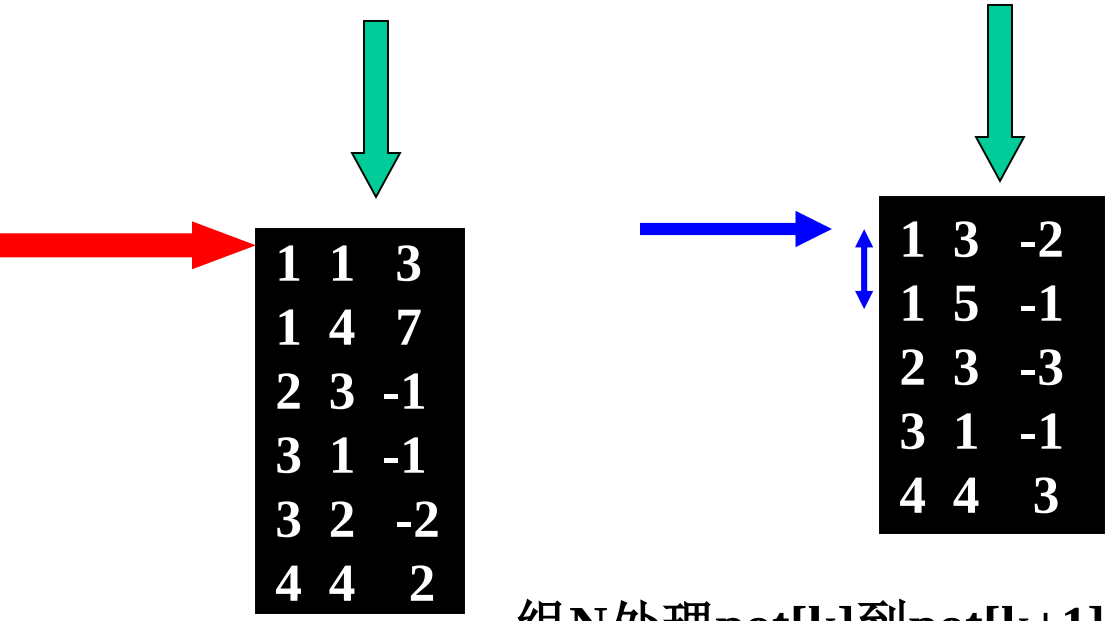
2）形成矩阵N各行的num, pot

row	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为N的行号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$



方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

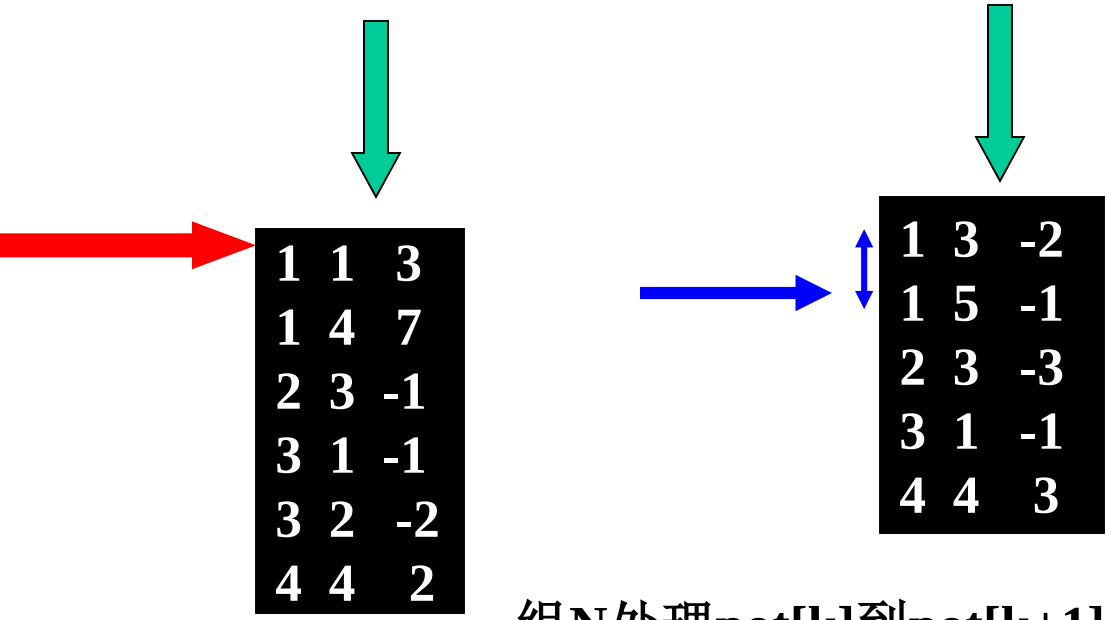
对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&0&0\\0&0&0&0&0\\0&0&0&0&0\\0&0&0&0&0\end{pmatrix}$$

3*(-1)

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

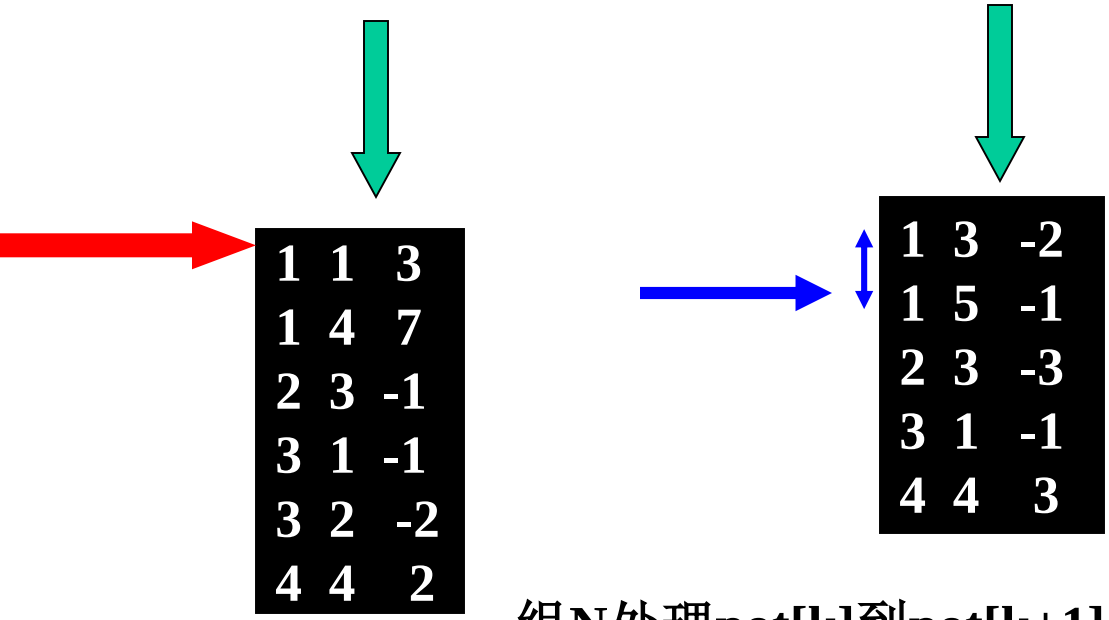
对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 0 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 0 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 0 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

7*3

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

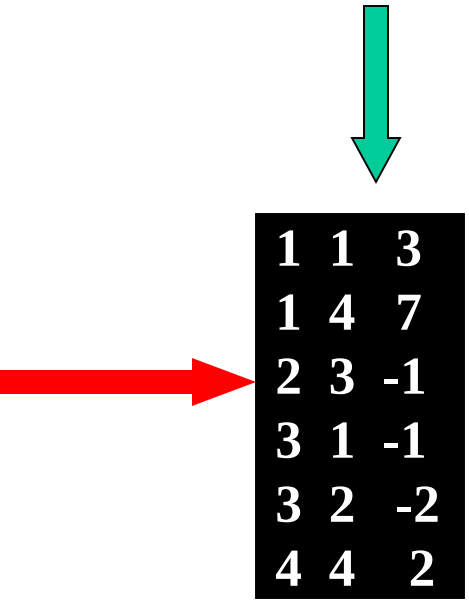
对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

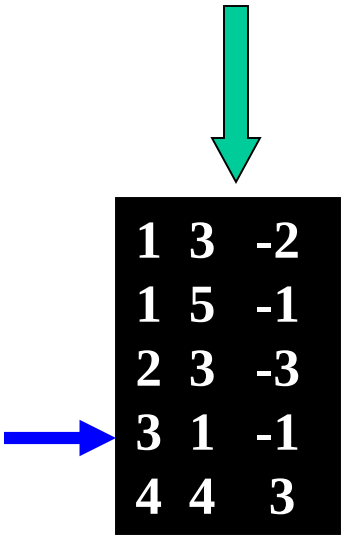
$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$





2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

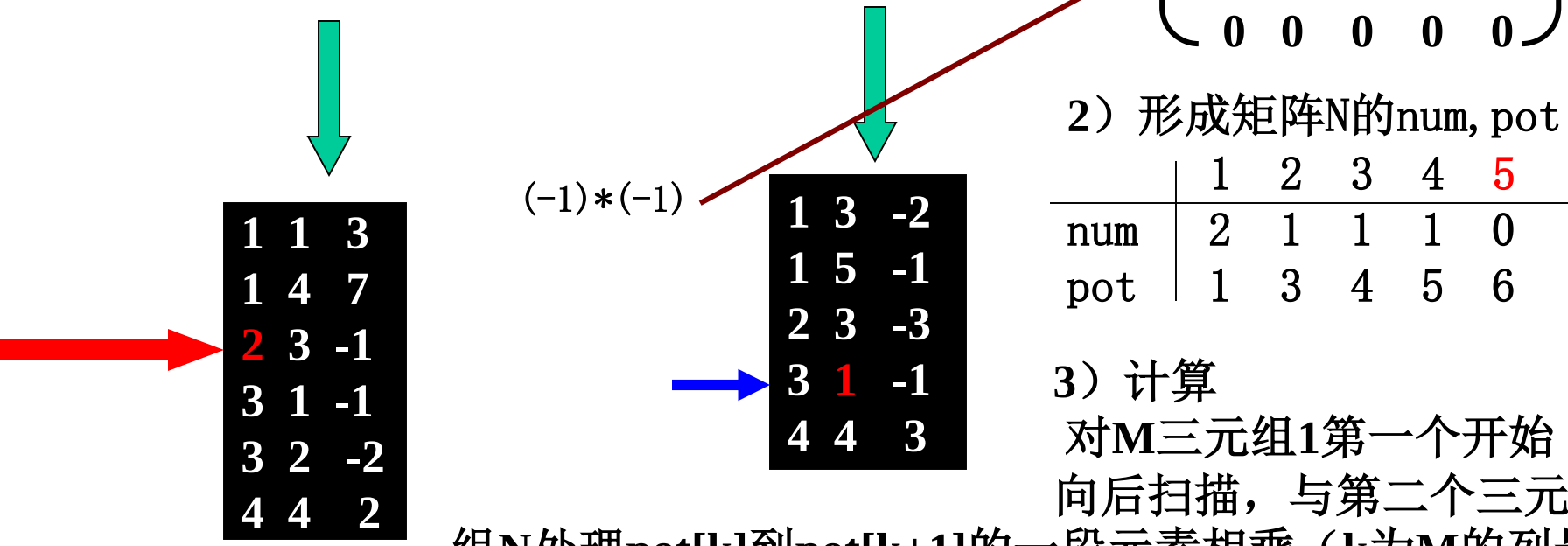
对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



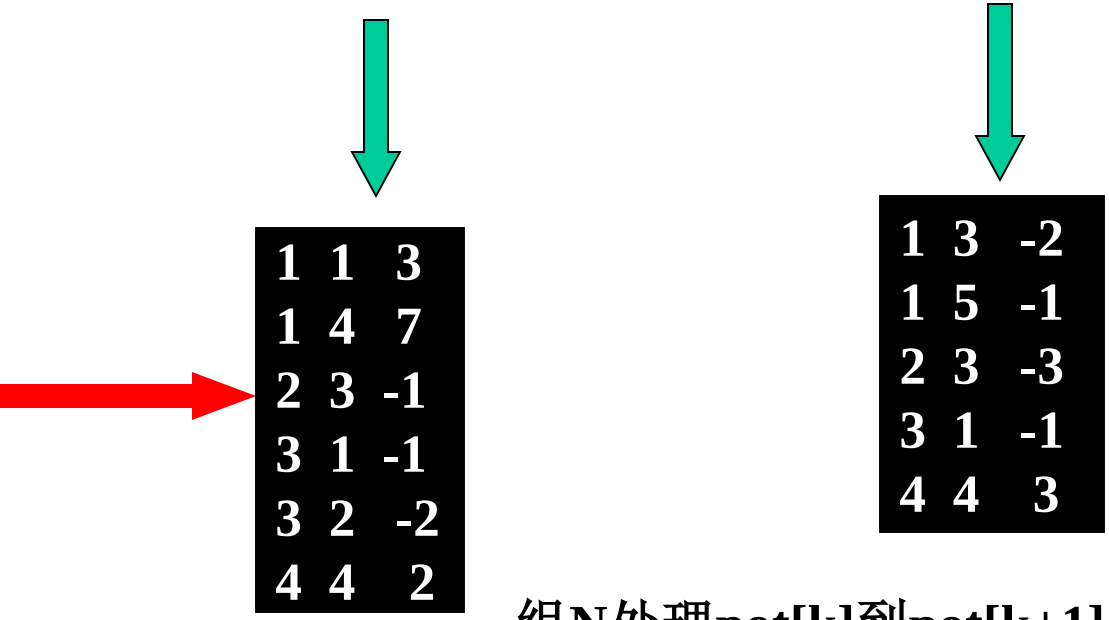
2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$



方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&0&0&0\\0&0&0&0&0\end{pmatrix}$$

−1*(−2)

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

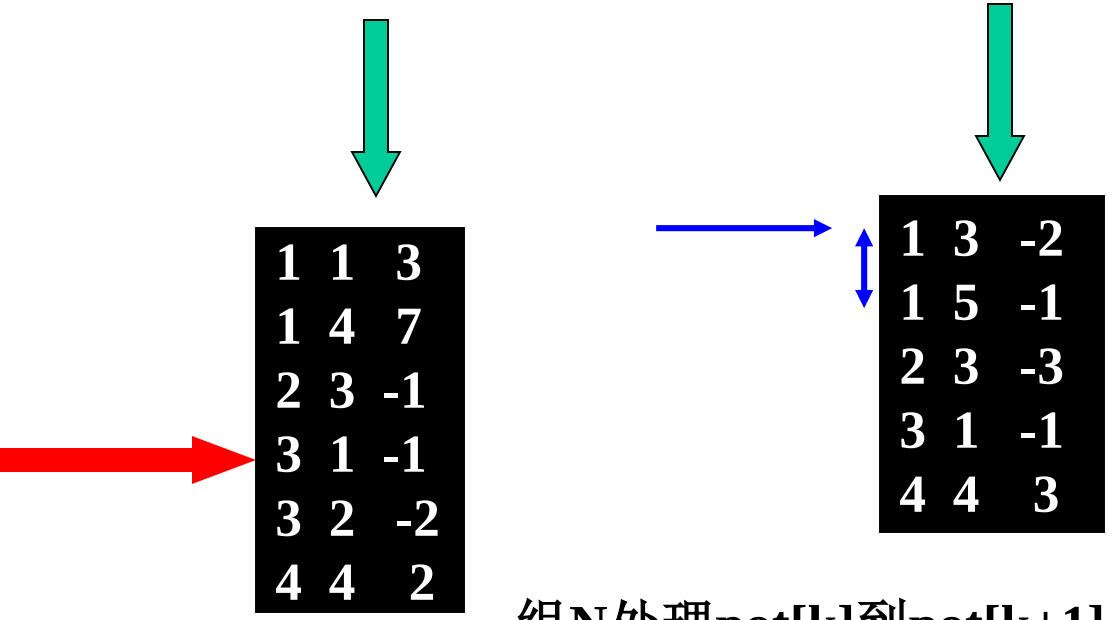
2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$



方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

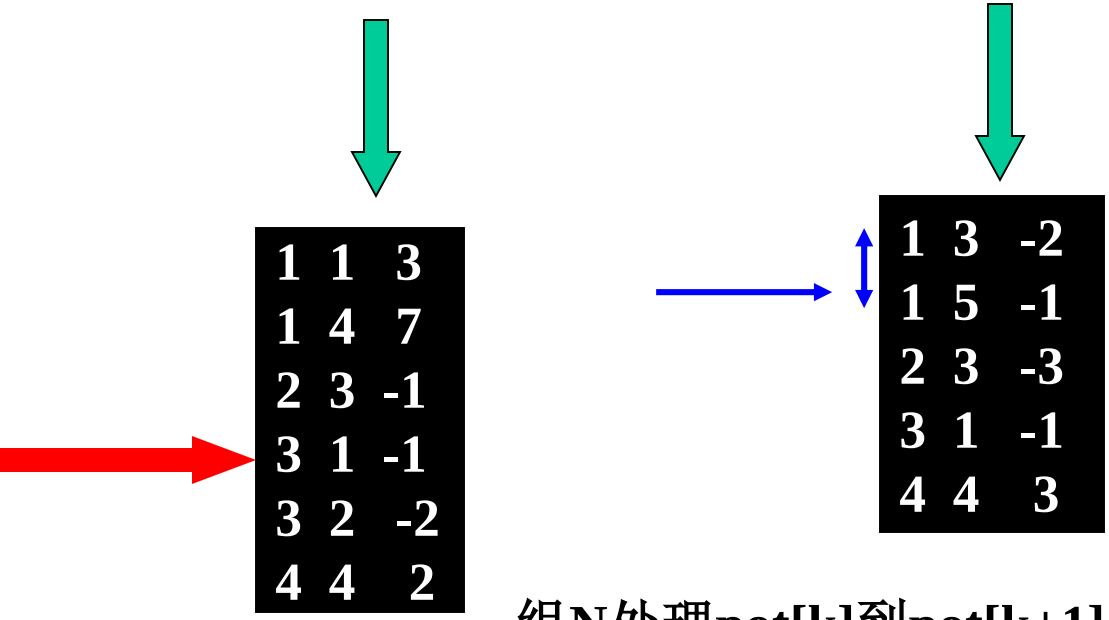
方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&0\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6



3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&0\\0&0&0&0&0\end{pmatrix}$$

-1*(-1)

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

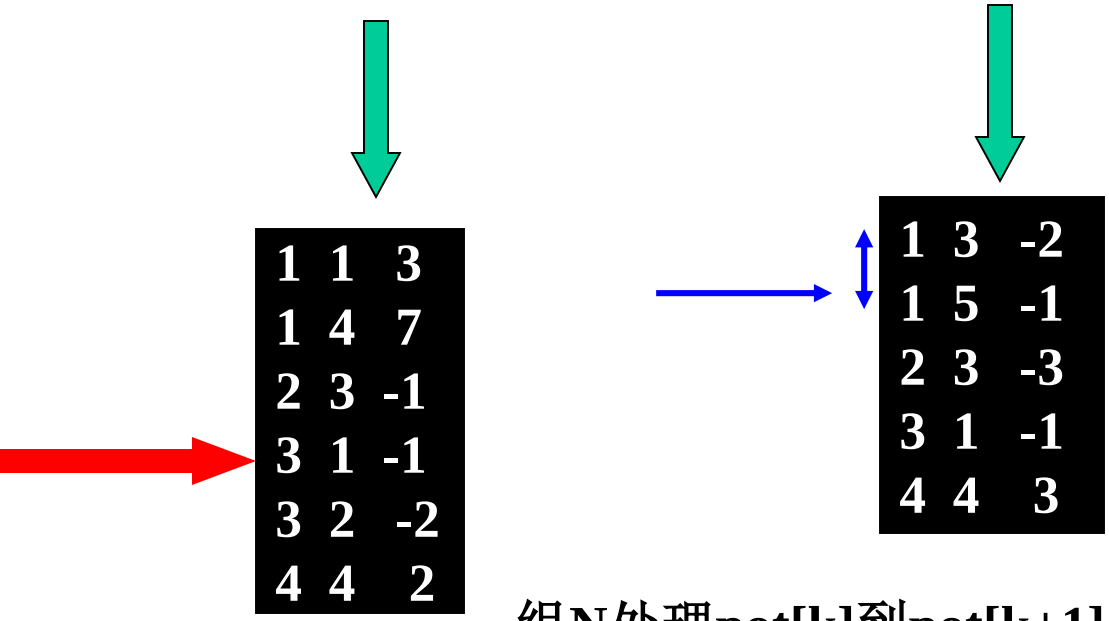
方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&1\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

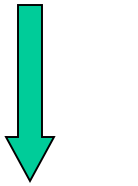


3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2



1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&1\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

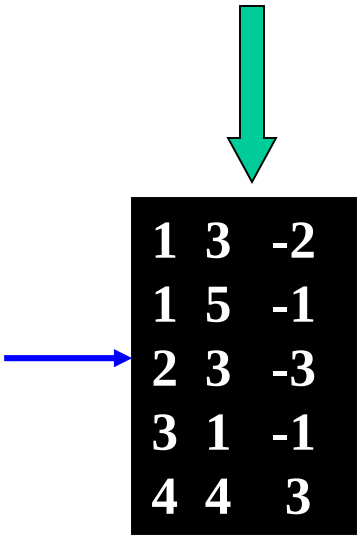
	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2



1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&1\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

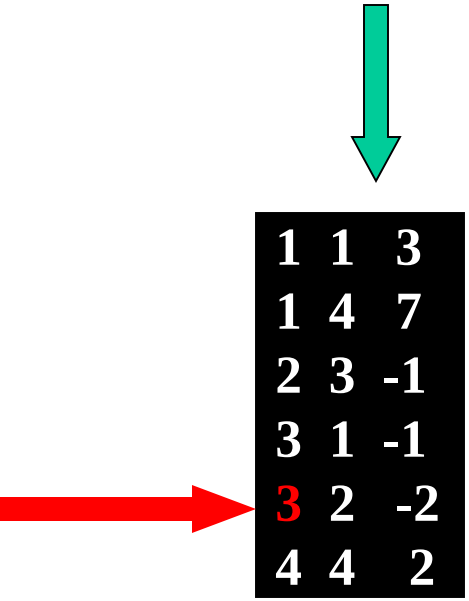
方法：

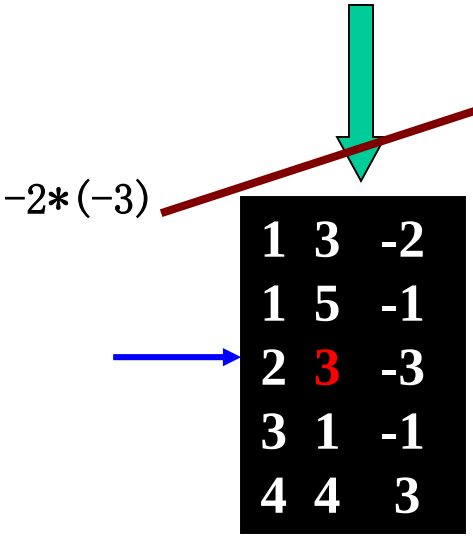
1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&2&0&1\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

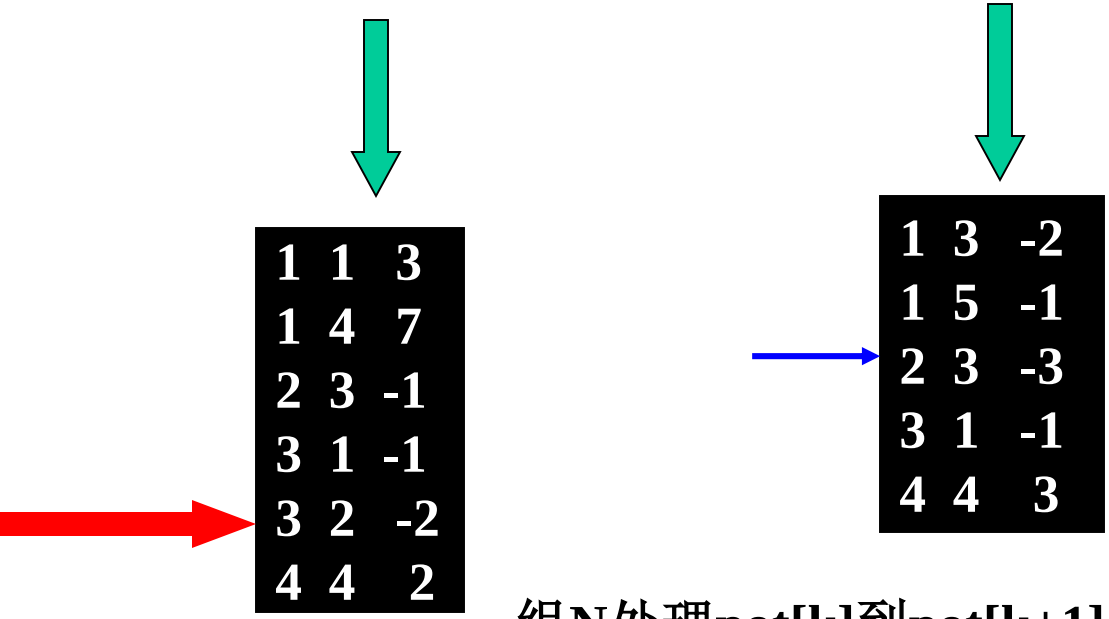




3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$



方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

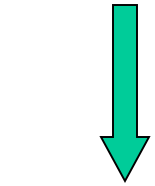
2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

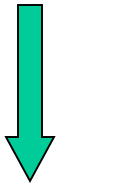
3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$



1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2



1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&8&0&1\\0&0&0&0&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ -1 & -2 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \quad N=\begin{pmatrix} 0 & 0 & -2 & 0 & -1 \\ 0 & 0 & -3 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 \end{pmatrix}$$

方法：

1）形成积矩阵

$$\begin{pmatrix} 0 & 0 & -6 & 21 & -3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

1	1	3
1	4	7
2	3	-1
3	1	-1
3	2	-2
4	4	2

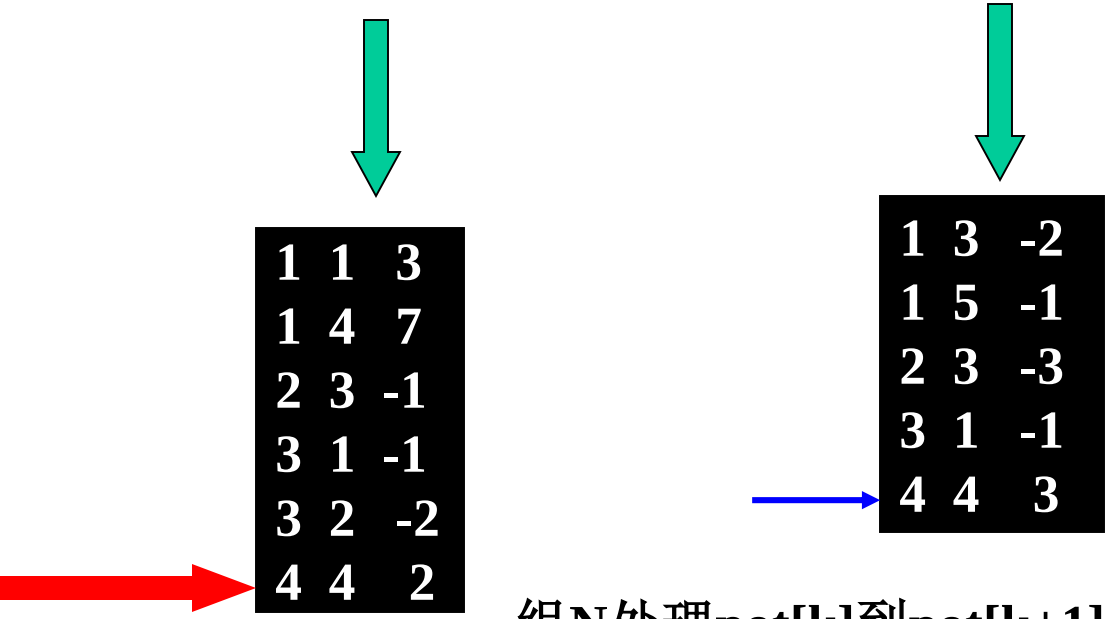
2*3

1	3	-2
1	5	-1
2	3	-3
3	1	-1
4	4	3

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

$$M=\begin{pmatrix}3&0&0&7\\0&0&-1&0\\-1&-2&0&0\\0&0&0&2\end{pmatrix}\qquad N=\begin{pmatrix}0&0&-2&0&-1\\0&0&-3&0&0\\-1&0&0&0&0\\0&0&0&3&0\end{pmatrix}$$



方法：

1）形成积矩阵

$$\begin{pmatrix}0&0&-6&21&-3\\1&0&0&0&0\\0&0&8&0&1\\0&0&0&6&0\end{pmatrix}$$

2）形成矩阵N的num, pot

	1	2	3	4	5
num	2	1	1	1	0
pot	1	3	4	5	6

3）计算

对M三元组1第一个开始向后扫描，与第二个三元组N处理pot[k]到pot[k+1]的一段元素相乘（k为M的列号）

十字链表

- 稀疏矩阵的加法 A = A+B

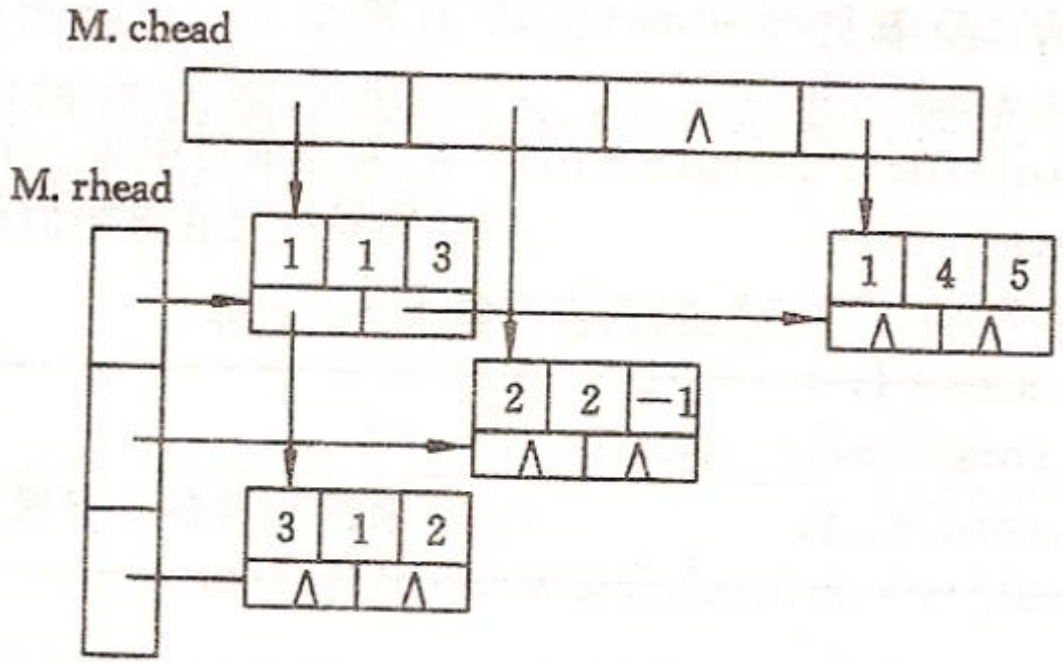


图 5.6　稀疏矩阵 M 的十字链表

```
Status CreateSMatrix_OL (CrossList &M) {
  // 创建稀疏矩阵 M。采用十字链表存储表示。
  if (M) free(M);
  scanf(&m, &n, &t );      // 输入 M 的行数、列数和非零元个数
  M.mu := m;   M.nu := n;   M.tu := t;
  if (!(M.rhead = (OLink *)malloc((m + 1) * sizeof(OLink))))exit(OVERFLOW);
  if (!(M.chead = (OLink *)malloc((n + 1) * sizeof(OLink)))) exit(OVERFLOW);
  M.rhead [ ] = M.chead [ ] = NULL ;      // 初始化行列头指针向量;各行列链表为空链表
  for (scanf(&i, &j, &e); i!= 0; scanf(&i, &j, &e)) {// 按任意次序输入非零元
    if (!(p = (OLNode *)malloc(sizeof(OLNode)))) exit(OVERFLOW);
    p->i = i;   p->j = j;   p->e = e;                    // 生成结点
    if (M.rhead[i] == NULL || M.rhead [i]->j>j)  {p->right = M.rhead [i];M.rhead [i] = p;}
    else {        // 寻查在行表中的插入位置
      for ( q = M.rhead [i]; (q->right) && q->right->j < j;  q = q->right );
      p->right = q->right;   q ->right = p;          } // 完成行插入
    if (M.chead [j] == NULL || M.chead[j]->i>i)  {p->down = M.chead [j]; M.chead [j] = p;}
    else {        // 寻查在列表中的插入位置
      for ( q = M.chead[j]; (q->down) && q->down->i < i;  q = q->down );
      p->down = q->down;   q->down = p;          }     // 完成列插入
}
```

典型题举例

- **1. 将一个A[1…100, 1…100]的三对角矩阵，按行优先顺序存入一维数组B[1…298]中，A中元素$a_{66,65}$（即该元素下标i = 66, j = 65）在B数组中的位置K为_____。**
- **A. 198　　B. 195　　C. 197**

三对角矩阵第一行和最后一行的非零元素是2个，其余行是3个；因此在a66, 65的前面一共有2+3*64=194个非零元素，而a66, 65则是第195个非零元素，故选B。

典型题举例

- 2. 下面_____属于特殊矩阵。
- A. 对角矩阵
- B. 上三角矩阵
- C. 下三角矩阵
- D. 稀疏矩阵
- E. 对称矩阵

正确答案：A、B、C、E

5.4 广义表的定义

广义表（Lists，又称列表）是线性表的推广。在第2章中，把线性表定义为n>=0个元素$a_1, a_2, a_3, \ldots, a_n$的有限序列。线性表的元素仅限于原子项，原子是作为结构上不可分割的成分，它可以是一个数或一个结构，若放松对表元素的这种限制，容许它们具有其自身结构，这样就产生了广义表的概念。

广义表是n(n>=0)个元素$a_1, a_2, a_3, \ldots, a_n$的有限序列，其中$a_i$或者是原子项，或者是一个广义表。通常记作LS=（$a_1, a_2, a_3, \ldots, a_n$）。LS是广义表的名字，n为它的长度。若$a_i$是广义表，则称它为LS的子表。

5.4 广义表的定义

ADT Glist {

数据对象： $D = \{e_i \mid i=1,2,..,n;\ n\geq 0;$

$e_i \in$ AtomSet 或 $e_i \in$ GList,

AtomSet为某个数据对象 }

数据关系：

$$LR = \{<e_{i-1}, e_i> \mid e_{i-1}, e_i \in D, 2\leq i\leq n\}$$

基本操作:

} ADT Glist

广义表是**递归**定义的**线性结构**，

$$LS = (\alpha_1, \alpha_2, \cdots, \alpha_n)$$

其中：α_i 或为原子 或为广义表

例如： A = ( )

F = (d, (e))

D = ((a,(b,c)), F)

C = (A, D, F)

B = (a, B) = (a, (a, (a, ⋯ ,)))

通常用圆括号将广义表括起来，用逗号分隔其中的元素。为了区别原子和广义表，书写时用大写字母表示广义表，用小写字母表示原子。若广义表

LS（n>=1)非空，则a_1是LS的表头，其余元素组成的表$(a_2, ...a_n)$称为LS的表尾。

广义表是递归定义的，因为在定义广义表时又用到了广义表的概念。广义表的例子如下：

（1）A=（）——A是一个空表，其长度为零。

（2）B=（e）——表B只有一个原子e，B的长度为1。

（3）C=（a,(b,c,d))——表C的长度为2，两个元素分别为原子a和子表(b,c,d)。

（4）D=（A，B，C）——表D的长度为3，三个元素都是广义表。显然，将子表的值代入后，则有

D=(( ),(e),(a,(b,c,d)))。

（5）E=（a,E）——这是一个递归的表，它的长度为2，

E相当于一个无限的广义表E=(a,(a,(a,(a,...)))).

从上述定义和例子可推出广义表的三个重要结论：

（1）广义表的元素可以是子表，而子表的元素还可以是子表。由此，广义表是一个多层次的结构，可以用图形象地表示。

（2）广义表可为其它表所共享。例如在上述例（4）中，广义表A，B，C为D的子表，则在D中可以不必列出子表的值，而是通过子表的名称来引用。

（3）广义表的递归性。

广义表是一个多层次的线性结构

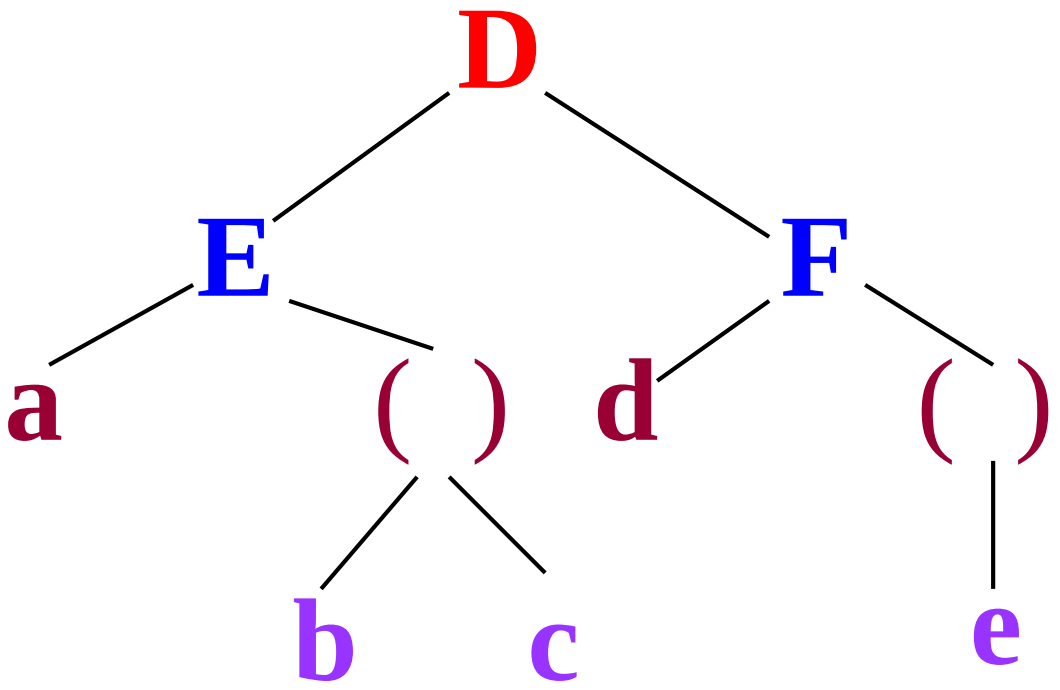
例如：

D=(E, F)

其中:

E=(a, (b, c))

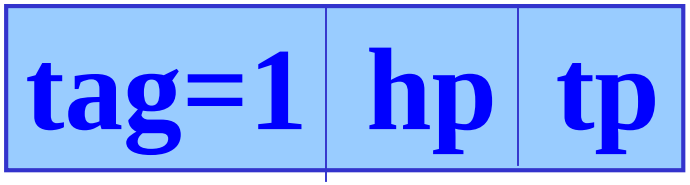
F=(d, (e))



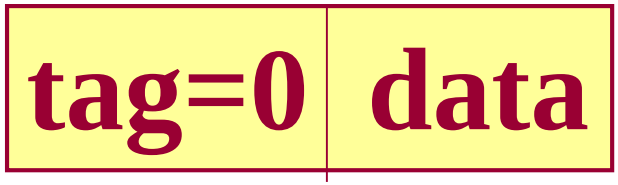
5.5 广义表的表示方法

通常采用头、尾指针的链表结构

表结点:



原子结点:

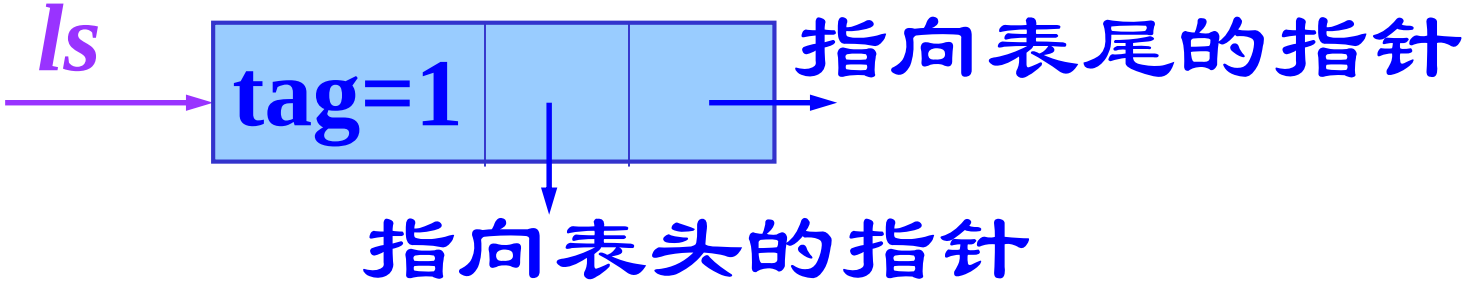


构造存储结构的两种方法:

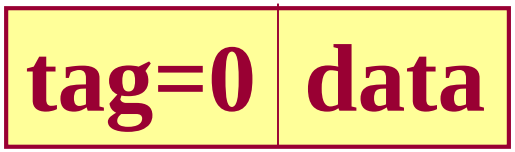
1) 头尾链表存储表示：

空表　　*ls=NIL*

非空表 *ls*



若表头为原子，则为



否则，依次类推。

(1) $A=(\ )$——A 是一个空表，它的长度为零。

(2) $B=(e)$——列表 B 只有一个原子 e，B 的长度为 1。

(3) $C=(a,(b,c,d))$——列表 C 的长度为 2

(4) $D=(A,B,C)$——列表 D 的长度为 3

(5) $E=(a,E)$——这是一个递归的表，它的长度为 2。

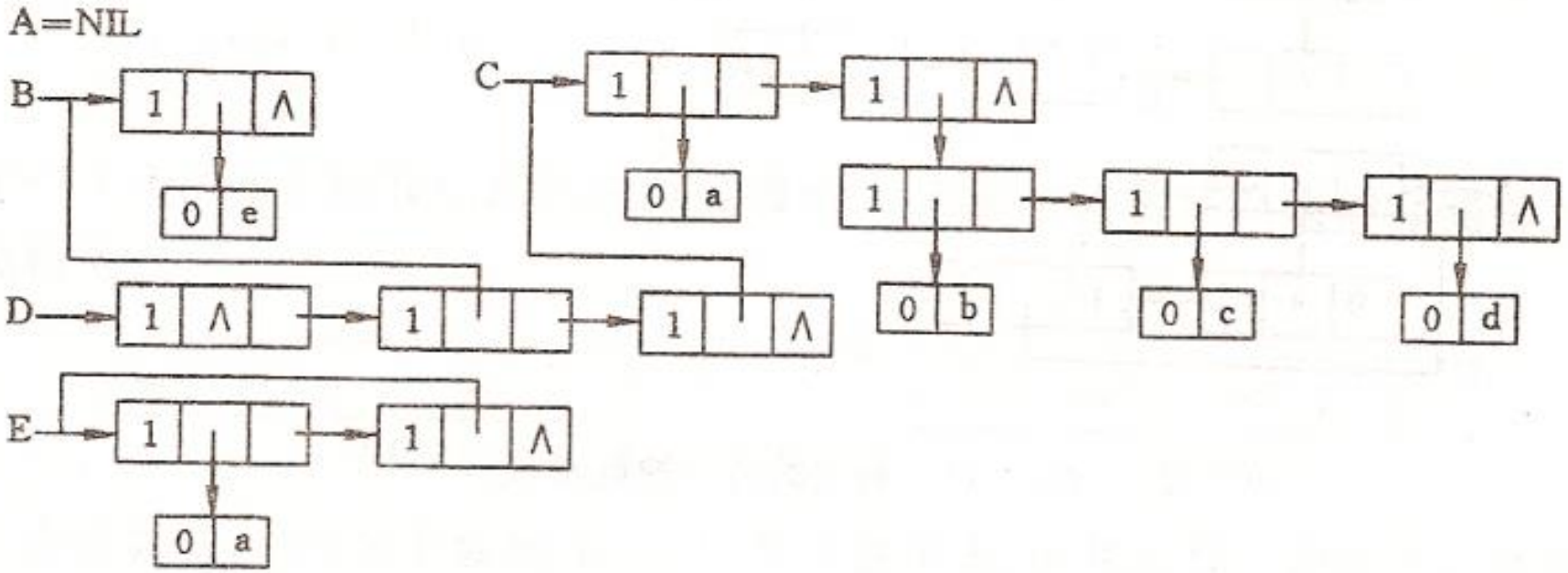


图 5.9　广义表的存储结构示例

本章小结

- 了解数组的两种存储表示方法，并掌握数组在以行为主的存储结构中的地址计算方法。
- 了解稀疏矩阵压缩存储方法的特点和适用范围，领会以三元组表示稀疏矩阵时进行矩阵运算采用的处理方法。
- 掌握广义表的结构特点及其存储表示方法，掌握两种结构的链表，学会对非空广义表进行分解的两种分析方法：即可将一个非空广义表分解为表头和表尾两部分或者分解为n个子表。